

The GMAP: A Versatile Tool for Physical Data Independence

Odysseas G. Tsatalos*

Marvin H. Solomon*

Yannis E. Ioannidis†

Computer Sciences Dept., Univ. of Wisconsin-Madison, WI 53706
{odysseas,solomon,yannis}@cs.wisc.edu

GMAP: 实现物理数据独立性的一种通用工具

Abstract

摘要

Physical data independence is touted as a central feature of modern database systems. Both relational and object-oriented systems, however, force users to frame their queries in terms of a logical schema that is directly tied to physical structures. Our approach eliminates this dependence. All storage structures are defined in a declarative language based on relational algebra as functions of a logical schema. We present an algorithm, integrated with a conventional query optimizer, that translates queries over this logical schema into plans that access the storage structures. We also show how to compile update requests into plans that update all relevant storage structures consistently and optimally. Finally, we report on experiments with a prototype implementation of our approach that demonstrate how it allows storage structures to be tuned to the expected or observed workload to achieve significantly better performance than is possible with conventional techniques.

物理数据独立性被誉为现代数据库系统的一项核心特性。然而，无论是关系系统还是面向对象系统，都要求用户基于逻辑模式来表述查询，而逻辑模式又与物理结构直接绑定。我们的方法消除了这种依赖：所有存储结构都用一种基于关系代数的声明式语言来定义，定义为逻辑模式的函数。我们提出了一个与传统查询优化器集成的算法，它能把针对该逻辑模式的查询翻译成访问这些存储结构的执行计划。我们还说明了如何把更新请求编译成执行

计划，以一致且最优的方式更新所有相关的存储结构。最后，我们报告了在该方法的原型实现上所做的实验。实验表明，这一方法允许根据预期或实测的负载来调整存储结构，所获得的性能明显优于传统技术所能达到的水平。

1 Introduction

1 引言

Physical data independence is usually described as the ability to write queries without being concerned with how the data are actually structured on disk. In current database systems (DBMSs), however, queries are tied to logical constructs such as relations, class extents, or object sets, that closely track the physical organization of data. In a relational database, for example, each relation is usually stored as a file, perhaps with a primary index. The database administrator can improve performance by adding secondary indices or by specifying the clustering of files, but more extensive improvements require modifying the logical schema, for example by de-normalizing tables. Such modifications necessitate rewriting queries and thus physical data independence is lost.

物理数据独立性通常被描述为：编写查询时无需关心数据在磁盘上究竟如何组织。然而在现有的数据库管理系统（DBMS）中，查询是与关系、类外延、对象集合这类逻辑结构绑定的，而这些逻辑结构又紧紧贴合着数据的物理组织。例如，在关系数据库中，每个关系通常存为一个文件，可能再加一个主索引。数据库管理员可以通过添加二级索引或指定文件的聚簇方式来提升性能，但更大幅度的改进就需要修改逻辑模式了，比如对表做反规范化。这类修改要求重写查询，物理数据独立性也就随之丧失。

Our goal is to improve physical data independence by decoupling physical decisions such as clustering and replication from the logical data model, so that the

physical organization can be altered without changing the logical schema or queries written against it. A more subtle benefit is that it places a wider range of possibilities for data organization at the disposal of the database administrator. For example, the fact that the data are described in a traditional normalized relational schema should not preclude a replicated, nested physical organization, if that organization would achieve better performance for the anticipated mix of queries and updates.

我们的目标是改善物理数据独立性：把聚簇、复制之类的物理决策与逻辑数据模型分离开，使得物理组织可以改变，而逻辑模式以及基于它编写的查询都不必改动。一个不那么显眼的好处是，数据库管理员在组织数据时有了更大的选择余地。例如，数据虽然是用传统的规范化关系模式描述的，但这不应妨碍采用一种带复制的嵌套物理组织——只要这种组织能让预期的查询与更新组合获得更好的性能。

Assume that data are stored in files of records, possibly implemented by an index structure such as a B⁺-tree. Instead of requiring a one-to-one correspondence between logical data constructs and physical storage structures (e.g., relation $\leftrightarrow$ file), we allow the contents of each file to be defined as a function of the logical schema, specified by a restricted relational-algebra expression. We call the combination of a file and its definition a *gmap* (pronounced gee-map and an acronym for Generalized Multilevel Access Path.) In the simplest cases, gmaps correspond to traditional storage structures such as an unordered file of the tuples in a relation or an index on that file. Gmaps, however, can also be used to partition the database vertically and horizontally and add multiple access paths, generalizing path indices. Since gmaps are allowed to contain overlapping data, they can also capture redundant

storage structures. Gmaps are invisible at the logical layer, so their definitions affect only the performance of queries and not their semantics.

假设数据存放在由记录组成的文件中，这些文件可能用 B+ 树之类的索引结构实现。我们不要求逻辑数据结构与物理存储结构一一对应（例如关系 $\leftrightarrow$ 文件），而是允许把每个文件的内容定义为逻辑模式的函数，用一个受限的关系代数表达式来描述。我们把“文件 + 它的定义”这一组合称为 gmap（读作 gee-map，是 Generalized Multilevel Access Path，即“广义多级访问路径”的缩写）。在最简单的情形下，gmap 就对应传统的存储结构，比如存放某个关系全部元组的无序文件，或建在该文件上的索引。不过，gmap 还可以用来对数据库做垂直和水平划分，以及添加多条访问路径，从而推广了路径索引。由于允许 gmap 包含相互重叠的数据，它们还能表达冗余的存储结构。gmap 在逻辑层是不可见的，因此它们的定义只影响查询的性能，不影响查询的语义。

In this paper, we restrict both gmap definitions and queries to project-select-join (psj) expressions over a simple semantic data model. We demonstrate that such expressions are powerful enough to express most conventional storage structures, as well as more “exotic” techniques such as path indices [2, 16], field replication [11, 23], and more. We present an algorithm to translate user queries, expressed as psj-queries over structures in the logical schema, into relational expressions over the gmaps. We also show how this translation can be integrated into a conventional query optimizer.

在本文中，我们把 gmap 的定义和查询都限制为一个简单语义数据模型之上的投影-选择-连接（psj）表达式。我们将说明，这类表达式的表达能力足以描述绝大多数传统存储结构，以及一些更“另类”的技术，如路径索引 [2, 16]、字段复制 [11, 23] 等。我们给出一个算法，把用户查询（即针对逻辑

模式中结构的 psj 查询) 翻译成针对 gmap 的关系表达式。我们还说明了如何把这一翻译过程集成到传统的查询优化器中。

***Partially supported by the Advanced Research Project Agency, ARPA order number 018 (formerly 8230), monitored by the U.S. Army Research Laboratory under contract DAAB07-91-C-Q518**

* 部分资助来自美国高级研究计划局 (ARPA), 订单号 018 (原 8230), 由美国陆军研究实验室监管, 合同号 DAAB07-91-C-Q518。

†Partially supported by grants from NSF (IRI-9113736, IRI-9224741, and IRI-9157368 (PYI Award)) DEC, IBM, HP, AT&T, and Informix.

† 部分资助来自 NSF (IRI-9113736、IRI-9224741 以及 IRI-9157368 (PYI 奖)), 以及 DEC、IBM、HP、AT&T 和 Informix。

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the VLDB copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Very Large Data Base Endowment. To copy otherwise, or to republish, requires a fee and/or special permission from the Endowment.

允许免费复制本材料的全部或部分内容, 前提是: 复制品不得用于直接商业利益的制作或分发; 须保留 VLDB 版权声明、出版物标题及其日期; 并注明复制已获得超大型数据库基金会 (Very Large Data Base Endowment) 的许可。以其他方式复制或再版, 须支付费用和/或获得该基金会的特别许可。

Proceedings of the 20th VLDB Conference Santiago, Chile, 1994

第 20 届 VLDB 会议论文集, 智利圣地亚哥, 1994

One of the benefits of our approach is that gmaps may store redundant data to improve the performance

of queries. Thus, updates may need to change multiple gmaps in a consistent manner. We show how a simple modification of the query translation algorithm can produce plans to perform these updates. We also demonstrate how this flexibility can be used in several other areas, e.g., acceleration of bulk loading of the database and acceleration of updates of complex objects.

我们的方法有一个好处：gmap 可以存储冗余数据来提升查询性能。这样一来，一次更新可能需要以一致的方式修改多个 gmap。我们将说明，只需对查询翻译算法稍加修改，就能生成执行这些更新的计划。我们还会展示这种灵活性如何用于其他一些方面，例如加速数据库的批量装载，以及加速复杂对象的更新。

All of the algorithms presented in this paper have been implemented in a prototype system. We report on experiments with a test database that illustrates that for a plausible mix of queries and updates, our techniques allow the physical representation to be tuned to provide better performance than what could be achieved through standard relational or object-oriented methods.

本文介绍的所有算法都已在一个原型系统中实现。我们在一组测试数据库上做了实验，结果表明：对于一组合理的查询与更新组合，我们的技术可以对物理表示进行调优，获得比标准的关系方法或面向对象方法更好的性能。

2 The Gmap Definition Language

2 gmap 定义语言

In this section, we introduce our data model and the corresponding data definition language. The data definition language (DDL) has two parts, the *logical DDL*, which defines the logical schema capturing the concep-

tual organization of the data, and the *physical DDL*, which defines the storage structures containing the data that instantiate the logical schema. We present the model in two notations, a semantic one (resembling the ER model) and a formal relational one. The two notations are equivalent; the semantic notation is more intuitive as a user interface, but all of our algorithms manipulate the relational forms of schemas.

本节介绍我们的数据模型及相应的数据定义语言。数据定义语言 (DDL) 分为两部分：逻辑 DDL 定义逻辑模式，刻画数据在概念上的组织；物理 DDL 定义存储结构，这些结构中存放的数据构成了逻辑模式的实例。我们用两种记法来表示这个模型：一种是语义记法（类似 ER 模型），另一种是形式化的关系记法。两种记法是等价的；语义记法作为用户界面更直观，但我们的所有算法操作的都是模式的关系形式。

2.1 The Logical Data Definition Language

2.1 逻辑数据定义语言

In the semantic notation, schemas are displayed as graphs. Throughout this paper we will illustrate our approach with an example database describing a university and its personnel (see Figure 1).

在语义记法中，模式以图的形式展示。本文始终用一个描述大学及其人员的示例数据库来说明我们的方法（见图 1）。

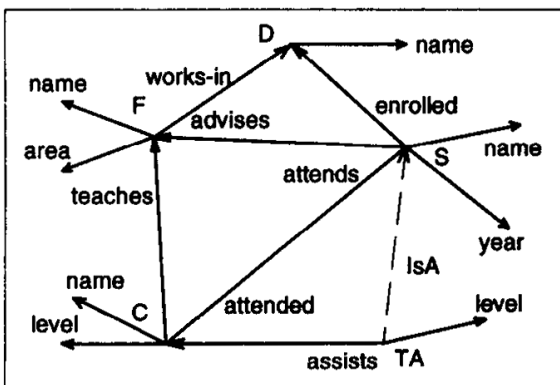


Figure 1: The logical schema

图 1: 逻辑模式

Nodes in this graph represent domains and solid edges represent relationships between them. Leaves represent primitive domains such as integers, character strings, or real numbers. Internal nodes represent domains populated with identity surrogates (tuple or object identifiers). In our example schema, these domains are Dept (department), Faculty, Student, Course, and TA (teaching assistant). To reduce clutter in the figures, these domain names are abbreviated to their initials. Functional dependencies are indicated by arrow heads. Inclusion dependencies (formally defined in Section 3) can also be expressed but are not shown for simplicity. IsA associations are denoted by dashed arcs pointing to the supertype. For our purposes, they are simply relationships with certain functional and inclusion dependencies implied by default. A name of the form $D.d$ is used to denote both a primitive domain and its relationship to an internal domain. For example, **Course.level** names both a primitive domain of integers and its relationship to the **Course** domain.

图中的节点表示域，实线边表示域之间的关系。叶节点表示基本域，如

整数、字符串或实数。内部节点表示由标识代理（元组标识符或对象标识符）构成的域。在我们的示例模式中，这些域是 Dept（系）、Faculty（教师）、Student（学生）、Course（课程）和 TA（助教）。为了让图不那么杂乱，这些域名在图中都缩写为首字母。函数依赖用箭头表示。包含依赖（第 3 节给出形式定义）也可以表达，但为简单起见没有画出来。ISA 关联用指向父类型的虚线弧表示。就本文而言，它们只是一种关系，默认隐含了某些函数依赖和包含依赖。形如 $D.d$ 的名字既表示一个基本域，也表示它与内部域 D 之间的关系。例如， Course.level 既指一个整数基本域，也指它与 Course 域之间的关系。

In the relational form of the data model, each edge of a schema graph from domain A to domain B is represented as a binary relation with attributes A and B . Because of this correspondence, we often use the term “attribute” to refer to domains (nodes in the graph) and “base relation” to refer to relationships (edges). Because our algorithms operate on the (binary) relational form of the schema, they apply to any semantic model that can be represented by binary relations with functional and inclusion dependencies.

在数据模型的关系形式中，模式图里每一条从域 A 指向域 B 的边都表示为一个以 A 、 B 为属性的二元关系。由于这种对应关系，我们常用“属性”一词指代域（图中的节点），用“基本关系”指代关系（图中的边）。由于我们的算法作用于模式的（二元）关系形式，它们适用于任何能用带函数依赖和包含依赖的二元关系来表示的语义模型。

2.2 The Physical Data Definition Language

2.2 物理数据定义语言

In our system, all physical storage structures are defined as gmaps. A gmap consists of a set of records (the *gmap data*), a query that indicates the seman-

tic relationships among the attributes of these records (the *gmap query*), and a description of the data structure used to store the records (the *gmap structure*). Although the actual database stores gmap data rather than the base relations, the gmap data may be thought of as the result of running the gmap query on the base relations.

在我们的系统中，所有物理存储结构都定义为 gmap。一个 gmap 由三部分组成：一组记录（gmap 数据）；一个查询，说明这些记录的各个属性之间的语义关系（gmap 查询）；以及对存储这些记录所用数据结构的描述（gmap 结构）。虽然数据库里实际存放的是 gmap 数据而不是基本关系，但可以把 gmap 数据看作在基本关系上运行 gmap 查询得到的结果。

Gmap queries are expressed in a simple SQL-like language. For example, the gmap

gmap 查询用一种类似 SQL 的简单语言来表达。例如，下面这个 gmap

```
def_gmap cs_faculty_by_area as btree by
  given Faculty.area
  select Faculty
  where Faculty works_in Dept and
         Dept = cs_oid
```

stores a set of pairs containing Faculty identifiers and the corresponding area names. Only faculty in the Computer Sciences department (identified by the constant `cs_oid`) are included. The gmap structure is a B^+ -tree indexed by `Faculty.area`. The entire `by` clause defines the gmap query. Attributes following the `given` and `select` keywords are called *input* and *output* attributes, respectively, and the selection `Dept = cs_oid` is called a *restriction*. Input attributes form the search key for gmap structures that allow associative access.

存储的是一组二元组，每个二元组包含一个 Faculty 标识符及其对应的

研究方向 (area) 名称。只有计算机科学系 (由常量 cs-oid 标识) 的教师才包含在内。该 gmap 的结构是一棵以 Faculty.area 为索引的 B+ 树。整个 by 子句定义了 gmap 查询。跟在关键字 given 和 select 后面的属性分别称为输入属性和输出属性, 选择条件 Dept = cs-oid 称为限制条件。对于支持按内容检索 (associative access) 的 gmap 结构, 输入属性构成其查找键。

The gmap query can also be expressed graphically as a subgraph of the schema graph called the *query graph* (see Figure 2). Shaded edges correspond to relationships explicitly mentioned in the where clause or implicitly mentioned as part of primitive attribute names. Input attributes are indicated by small arrows, and output attributes are indicated by double circles around nodes. Restrictions are described as annotations on the corresponding nodes.

gmap 查询也可以用图形方式表示为模式图的一个子图, 称为查询图 (见图 2)。阴影边对应 where 子句中显式提到的关系, 或者作为基本属性名的一部分被隐式提到的关系。输入属性用小箭头标出, 输出属性用节点外的双圆圈标出。限制条件以注记的形式标在相应节点上。

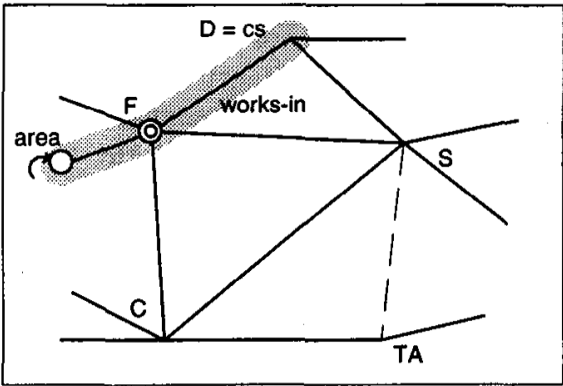


Figure 2: A collection based index

图 2: 基于集合的索引

Each query expressible in this language is equivalent to a restricted project-select-join (psj) query on the relational form of the logical schema :

用这种语言可以表达的每个查询，都等价于逻辑模式关系形式上的一个受限的投影-选择-连接（psj）查询：

$$Q = \pi_{AS}(\sigma_S(R_1 \bowtie R_2 \bowtie \dots \bowtie R_n)).$$

In this example,

在这个例子中，

$$Q = \pi_{F, F.area} \sigma_{D=cs_oid}(F.area \bowtie works_in).$$

Expressible queries obey three restrictions:

- They are range-restricted, i.e., all attributes in S and A are attributes of the relations R_i ,
- selections are conjunctions of comparisons ($=, >, \geq, <, \leq$) between attributes and constants, and
- joins are natural, i.e., only attributes with the same name are joined and all attributes maintain their name in the result.

可表达的查询须满足三个限制：

- 它们取值域受限的，即 S 和 A 中的所有属性都是关系 R_i 的属性；
- 选择条件是属性与常量之间比较（ $=, >, \geq, <, \leq$ ）的合取；
- 连接都是自然连接，即只有同名属性才相互连接，并且所有属性在结果中保留原名。

In the rest of the paper, we use the term *psj-query* to refer to a query that conforms to these restrictions.

在本文余下部分，我们用 psj 查询（psj-query）一词来指满足上述限制的查询。

2.3 The Query Language

2.3 查询语言

We often use the term *logical query* to refer to queries posed on the logical schema. We currently require logical queries to be written in the same language we use for gmap queries. That is, they must be restricted psj-queries. In addition, each query must be translatable into a psj-query over gmaps or projections of them. Thus, we do not handle cases where logical queries need to be translated into unions or arbitrary sequences of psj-queries. Note that translated logical queries involve relations with arbitrary arity (the gmap data), while gmap queries involve binary relations only (corresponding to relationships).

我们常用“逻辑查询”一词指针对逻辑模式提出的查询。目前我们要求逻辑查询用与 gmap 查询相同的语言来编写，也就是说，它们必须是受限的 psj 查询。此外，每个查询必须能翻译成一个针对 gmap（或其投影）的 psj 查询。因此，对于那些需要翻译成若干 psj 查询的并、或任意 psj 查询序列的逻辑查询，我们不做处理。注意，翻译后的逻辑查询涉及的是任意元数的关系（即 gmap 数据），而 gmap 查询只涉及二元关系（对应于模式中的关系）。

2.4 Examples

2.4 示例

Gmaps can be used to define arbitrary physical representations, including those of a conventional normalized relational database, an object-oriented database, or any combination of the two.

gmap 可以用来定义任意的物理表示，包括传统规范化关系数据库的物理表示、面向对象数据库的物理表示，或者两者的任意组合。

To illustrate the object-oriented approach, suppose we want to cluster together all information about each faculty member. Given the object identifier of a **Faculty** object, we should be able to retrieve personal information as well as the object identifiers of the faculty member's department, advisees, and courses taught. A gmap that meets these specifications may be defined and drawn as in Figure 3.

为了说明面向对象的做法，假设我们想把每位教师的所有信息聚簇在一起。给定一个 **Faculty** 对象的对象标识符，我们应该能够取出该教师的个人信息，以及其所在系、所指导学生和所授课程的对象标识符。满足这些要求的 gmap 可以按图 3 那样定义和绘制。

```
def_gmap faculty_data as heap by
  given Faculty select Student, Dept,
    Course, Faculty.area, Faculty.name
  where Faculty works_in Dept and
    Faculty advises Student and
    Faculty teaches Course
```

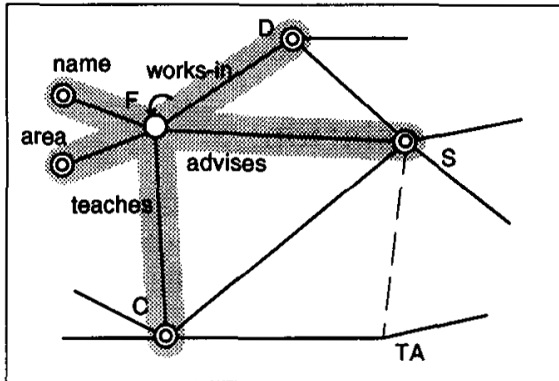


Figure 3: The Faculty class extent

图 3: Faculty 类外延

A secondary index in a relational system can also be defined easily in our language. For example, an index on the faculty area is defined as in Figure 4.

关系系统中的二级索引也可以方便地用我们的语言来定义。例如，教师研究方向（area）上的索引可以按图 4 那样定义。

```
def_gmap faculty_index_on_area as btree by
given Faculty.area select Faculty
```

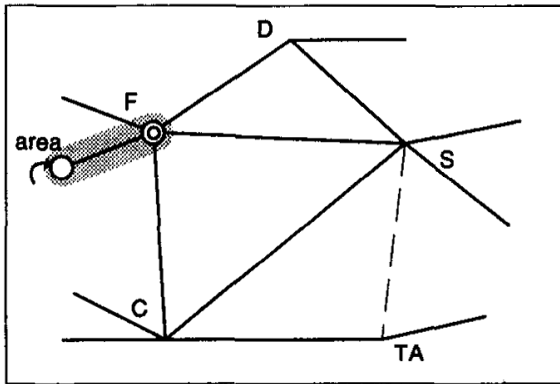


Figure 4: A Faculty index on "area"

图 4: Faculty 在 "area" 上的索引

Note that the index is not defined in terms of the previous gmap, as would be the case in a relational database, but in terms of the logical schema.

注意，这个索引并不是像在关系数据库中那样基于前一个 gmap 来定义的，而是基于逻辑模式来定义的。

In the `faculty_data` example, it might be desirable to include in a faculty member's record the department name in addition to the department id, because for example, the department name is frequently printed along with the name of the faculty member. The department name is in this case a *nested attribute*

of the Faculty domain. This essentially implements *field replication* [11, 23], which has been shown to offer several advantages. The only change necessary is to add “Dept.name” to the **select** clause.

在 faculty_data 这个例子中，也许希望在教师记录中除了系的 id 之外再存上系的名称，比如因为系名经常要和教师姓名一起打印出来。这时，系名就是 Faculty 域的一个嵌套属性。这实际上就是在实现字段复制 [11, 23]，已有研究表明这种技术有好几方面的优点。所需的唯一改动，就是在 select 子句中加上 “Dept.name”。

In the previous examples, the gmap data included all Faculty instances. However, there are cases where we frequently access only some instances of a domain. Object-oriented systems that store instances in explicit collections rather than class extents [5, 16, 18] allow the creation of collection indices, which provide fast access paths only to the subsets of the domains that are included in the collection. Our gmap definition language is powerful enough to express such indices by using restrictions. An example of this technique was shown in Figure 2 above.

在前面的例子中，gmap 数据包含了所有 Faculty 实例。然而，有些情况下我们只会频繁访问某个域中的一部分实例。有些面向对象系统把实例存放在显式的集合（collection）中，而不是类外延中 [5, 16, 18]，这类系统允许建立集合索引，只为域中包含在该集合里的那部分实例提供快速访问路径。我们的 gmap 定义语言借助限制条件，足以表达这类索引。前面的图 2 就是这种技术的一个例子。

Many more indexing schemes can be specified using gmaps, e.g., nested indices, replication of non-functional nested attributes, and indices with composite keys where each key component is a path. A complete taxonomy of existing indexing schemes and

other advanced storage structures that can be defined by using gmaps is presented elsewhere [24].

用 gmap 还可以描述许多其他索引方案，例如嵌套索引、非函数型嵌套属性的复制，以及复合键索引（其中每个键分量都是一条路径）。关于现有索引方案以及其他可以用 gmap 定义的高级存储结构，完整的分类见 [24]。

3 Query translation

3 查询翻译

Before presenting the actual translation algorithm, we first introduce some additional notation and definitions, and also discuss two auxiliary problems that arise as part of query translation.

在介绍具体的翻译算法之前，我们先引入一些额外的记号和定义，并讨论查询翻译过程中出现的两个辅助问题。

3.1 Notation

3.1 记号

For convenience, we use a triplet $\langle Q_r, Q_s, Q_p \rangle$ as an alternative way of representing a psj-query Q . The set Q_r contains the joined binary relations, the set Q_s contains the selection predicates, and the set Q_p contains the projected attributes. We call the set Q_p the *query target*, and its members *target attributes*. Given a query Q , we frequently deal with its part that includes only relations in a set $\mathcal{R}$, denoted $Q[rel \in \mathcal{R}]$. Similarly, the subset of Q_s that mentions only attributes in a set $\mathcal{A}$ is denoted $Q_s[attr \in \mathcal{A}]$. The set of attributes in a set of relations $\mathcal{R}$ is denoted $A(\mathcal{R})$.

为方便起见，我们用三元组 $\langle Q_r, Q_s, Q_p \rangle$ 作为 psj 查询 Q 的另一种表示方式。集合 Q_r 包含参与连接的二元关系，集合 Q_s 包含选择谓词，集合 Q_p 包含

投影属性。我们称集合 Q_p 为查询目标，称其中的元素为目标属性。给定查询 Q ，我们经常要处理它只涉及集合 $\mathcal{R}$ 中关系的那一部分，记作 $Q[\text{rel} \in \mathcal{R}]$ 。类似地， Q_s 中只涉及集合 $\mathcal{A}$ 中属性的那部分子集记作 $Q_s[\text{attr} \in \mathcal{A}]$ 。关系集合 $\mathcal{R}$ 中所有属性构成的集合记作 $A(\mathcal{R})$ 。

3.2 Definitions

3.2 定义

The *natural join* of two psj-queries P and Q , denoted $P \bowtie Q$, is the natural join of their result relations. The *add-join* of two psj-queries P and Q , denoted $P \oplus Q$, is the psj-query $P \oplus Q = \langle P_r \cup Q_r, P_s \cup Q_s, P_p \cup Q_p \rangle$. The add-join differs from the natural join in that the projections of P and Q are performed *after* all joins of base relations rather than being interleaved with them.

两个 psj 查询 P 和 Q 的自然连接，记作 $P \bowtie Q$ ，是它们结果关系的自然连接。两个 psj 查询 P 和 Q 的加连接（add-join），记作 $P \oplus Q$ ，是 psj 查询 $P \oplus Q = \langle P_r \cup Q_r, P_s \cup Q_s, P_p \cup Q_p \rangle$ 。加连接与自然连接的区别在于： P 和 Q 的投影是在所有基本关系的连接都完成之后才执行的，而不是穿插在这些连接之间。

Let $R_1(\alpha, \beta), R_2(\beta, \gamma)$ be two relations with a common attribute β . An *inclusion dependency* from R_1 to R_2 exists, denoted $R_1.\beta \subseteq R_2.\beta$, if every value of β in R_1 appears also in R_2 .

设 $R_1(\alpha, \beta)$ 、 $R_2(\beta, \gamma)$ 是两个具有公共属性 β 的关系。如果 R_1 中 β 的每个值都在 R_2 中出现，则称存在一个从 R_1 到 R_2 的包含依赖，记作 $R_1.\beta \subseteq R_2.\beta$ 。

Multivalued dependencies are not meaningful in binary relations, but are important in n-ary results of psj-queries, such as the gmap data. Since lossless join

decompositions imply multivalued dependencies and gmap data are the result of joining base relations, we can infer certain multivalued dependencies for each gmap. An algorithm that generates a cover of the multivalued dependencies that hold on the gmap relation is described in a longer version of this paper [25]. Multivalued dependencies are important because they help determining the pieces of the gmap relation that can be used to answer a user query.

多值依赖对二元关系没有意义，但对 psj 查询的 n 元结果（如 gmap 数据）却很重要。由于无损连接分解蕴含多值依赖，而 gmap 数据又是基本关系连接的结果，我们可以为每个 gmap 推导出某些多值依赖。在 gmap 关系上成立的多值依赖，其覆盖可由一个算法生成，该算法在本文的完整版 [25] 中给出。多值依赖之所以重要，是因为它们有助于确定 gmap 关系的哪些部分可以用来回答用户查询。

3.3 Query Equivalence

3.3 查询等价

When translating a logical query into a query over gmaps, we often need to test the equivalence of psj-queries. Two psj-queries Q_1 and Q_2 are equivalent, denoted $Q_1 \equiv Q_2$, if they produce the same result for any valid instance of the database schema. Equivalence testing of arbitrary conjunctive queries, even without taking into account any dependencies, is NP-complete [1, 7]. On the other hand, we can efficiently compare two psj-queries syntactically to see if they are identical (up to trivial differences such as the ordering of the join terms). This is a sufficient condition for equivalence, which we use in our translation algorithm. We are also interested in two special cases of equivalence testing, where psj-queries of specific forms are involved and various types of dependencies are taken

into account. These are discussed in the next two subsections, where sufficient conditions for equivalence are provided.

把逻辑查询翻译成针对 gmap 的查询时，我们经常需要检验 psj 查询是否等价。如果两个 psj 查询 Q_1 和 Q_2 对数据库模式的任何合法实例都产生相同的结果，就称它们等价，记作 $Q_1 \equiv Q_2$ 。对任意合取查询做等价检验，即使不考虑任何依赖，也是 NP 完全的 [1, 7]。另一方面，我们可以高效地在语法上比较两个 psj 查询，看它们是否完全相同（不计连接项的次序之类的细微差别）。这是等价的一个充分条件，我们的翻译算法就用它。我们还关心等价检验的两种特殊情形，其中涉及特定形式的 psj 查询，并且要考虑各类依赖。下面两小节讨论这两种情形，并给出等价的充分条件。

3.3.1 Coverage

3.3.1 覆盖

A query Q *covers* a set of relations $\mathcal{R}$ if

如果下式成立，就称查询 Q 覆盖关系集合 $\mathcal{R}$ ：

$$Q[\text{rel} \in \mathcal{R}] \equiv \pi_{A(\mathcal{R})}(Q) \quad (1)$$

For example, if $R_1(\alpha, \beta) \equiv \pi_{\alpha, \beta}(R_1(\alpha, \beta) \bowtie R_2(\beta, \gamma))$ then $R_1 \bowtie R_2$ covers $\{R_1\}$. In general, the result of the left-hand side query is a superset of that of the right-hand side query. When (1) holds, the part of the query that involves relations not in $\mathcal{R}$ (relation R_2 in our example) has no effect on $\pi_{A(\mathcal{R})}(Q)$, in the sense that it does not filter out any tuples produced by the rest of the query. An algorithm that implements a sufficient condition for testing coverage is presented elsewhere [25]. The algorithm makes use of the inclusion dependencies of the schema. Its running time is linear in the size of Q and quadratic in the number of inclusion dependencies between relations in Q .

例如, 若 $R_1(\alpha, \beta) \equiv \pi_{\alpha, \beta}(R_1(\alpha, \beta) \bowtie R_2(\beta, \gamma))$, 则 $R_1 \bowtie R_2$ 覆盖 $\{R_1\}$ 。一般而言, 左边查询的结果是右边查询结果的超集。当 (1) 成立时, 查询中涉及 $\mathcal{R}$ 之外关系的那部分 (本例中即关系 R_2) 对 $\pi_{A(\mathcal{R})}(Q)$ 没有影响, 也就是说, 它不会过滤掉查询其余部分产生的任何元组。检验覆盖的一个充分条件由一个算法实现, 该算法见 [25]。它利用了模式中的包含依赖, 运行时间与 Q 的规模成线性关系, 与 Q 中各关系之间包含依赖的个数成平方关系。

3.3.2 Natural join vs. Add-Join

3.3.2 自然连接与加连接

In general, if P and Q are two psj-queries, $P \oplus Q \subseteq P \bowtie Q$. However, in the presence of certain integrity constraints $P \oplus Q \equiv P \bowtie Q$. For example, suppose $R(\alpha, \beta), S(\beta, \gamma)$ are two relations, P is the query $\pi_{\alpha\beta}(R) = R$, and Q is the query $\pi_{\alpha\gamma}(R \bowtie S)$. Then $P \oplus Q$ is $\pi_{\alpha\beta\gamma}(R \bowtie R \bowtie S) = R \bowtie S$, which is not in general the same as $R \bowtie \pi_{\alpha\gamma}(R \bowtie S)$. If, however, β is functionally determined by α in R (that is, α is a key for R), the two joins are equal. Intuitively, the information “lost” by projecting away the β attribute in $P \bowtie Q$ can be completely recovered from the remaining α attributes.

一般来说, 若 P 和 Q 是两个 psj 查询, 则 $P \oplus Q \subseteq P \bowtie Q$ 。但在某些完整性约束下, $P \oplus Q \equiv P \bowtie Q$ 。例如, 设 $R(\alpha, \beta), S(\beta, \gamma)$ 是两个关系, P 是查询 $\pi_{\alpha\beta}(R) = R$, Q 是查询 $\pi_{\alpha\gamma}(R \bowtie S)$ 。那么 $P \oplus Q$ 就是 $\pi_{\alpha\beta\gamma}(R \bowtie R \bowtie S) = R \bowtie S$, 它一般不同于 $R \bowtie \pi_{\alpha\gamma}(R \bowtie S)$ 。但如果在 R 中 β 由 α 函数决定 (即 α 是 R 的键), 这两个连接就相等。直观地说, 在 $P \bowtie Q$ 中因投影掉 β 属性而“丢失”的信息, 可以完全由剩下的 α 属性恢复出来。

Detecting when the natural join of two psj-queries is equivalent to a psj-query is very important in our query translation algorithm, since it allows us to

rewrite the join of two gmaps (which are psj-queries) as a psj-query. The algorithm iteratively performs such rewritings in order to express the join of several gmaps as a psj-query, which is then checked syntactically for equivalence with the user query. As a sufficient condition for guaranteeing that the natural join of two queries is a psj-query, we test if it is equivalent to the add-join of the queries in question. An algorithm that implements a sufficient condition for testing this equivalence is presented elsewhere [25]. The algorithm makes use of multivalued dependencies and of query coverage and its running time is quadratic in the size of queries and in the size of multivalued dependencies.

判断两个 psj 查询的自然连接何时等价于某个 psj 查询，在我们的查询翻译算法中非常重要，因为这样就能把两个 gmap（它们都是 psj 查询）的连接改写成一个 psj 查询。算法反复进行这种改写，把若干个 gmap 的连接表示成一个 psj 查询，然后在语法上检验它与用户查询是否等价。为了保证两个查询的自然连接是 psj 查询，我们采用的充分条件是：检验它是否等价于这两个查询的加连接。实现这一充分条件的算法见 [25]。该算法利用了多值依赖和查询覆盖，运行时间与查询规模及多值依赖规模成平方关系。

3.4 Query Translation Algorithm

3.4 查询翻译算法

Below we present an algorithm to translate a logical psj-query into a query over gmaps. To simplify the presentation, we omit most considerations of efficiency.

下面给出一个算法，把逻辑 psj 查询翻译成针对 gmap 的查询。为了便于说明，我们略去了大部分效率方面的考虑。

2); and the gmap must cover the common relations with the query (otherwise, the gmap will not have all the information needed by the query) (line 3).

算法首先把搜索空间缩小到与查询有关的 gmap (第 1-3 行)。具体来说: gmap 必须至少与查询有一个共同的关系, 并且该关系至少有一个属性包含在 gmap 的结果中 (第 1 行); 查询中针对 gmap 所含关系的属性的选择条件, 要么作用在 gmap 的目标属性上 (这样才能施加到这些属性上), 要么必须与 gmap 自身的选择条件完全相同 (第 2 行); 此外, gmap 必须覆盖它与查询共有的那些关系, 否则 gmap 就不具备查询所需的全部信息 (第 3 行)。

Each possible subset of the relevant gmaps (line 4) gives rise to a single candidate translation (assuming that selections are always pushed through the joins):

相关 gmap 的每个可能子集 (第 4 行) 都对应一个候选翻译 (假定选择条件总是被下推穿过连接):

$$\pi_{Q_p}(\pi_{A(Q_r)}\sigma_{Q_s}G_1 \bowtie \cdots \bowtie \pi_{A(Q_r)}\sigma_{Q_s}G_n). \quad (2)$$

The rest of the algorithm tests whether or not this query expression is equivalent to the given logical query. Each join operand in the above equation is a projection and a selection on a gmap. Since we verified earlier (line 2) that the query selections can be pushed through the gmap projections, the join operands are psj-queries. The algorithm tries to express their join as a psj-query as well. The join operands are scanned (line 5) and any pair whose natural join is equivalent to their add-join (line 6) is replaced by a single join operand which is again a psj-query (line 7). The set of join operands thus keeps reducing. At some point, we can no longer reduce the set either because there is just one psj-query left or because there is no pair that

satisfies the equivalence test (line 6). In the former case, the remaining psj-query is equivalent to the initial join expression (2) after performing one final projection step (π_{Q_p}) and can be syntactically checked for equivalence (line 8) with the logical query. In the latter case, the subset chosen in line 4 is rejected. Algorithm 1 satisfies the following.

算法其余部分检验这个查询表达式是否与给定的逻辑查询等价。上式中每个连接操作数都是对某个 gmap 做的投影和选择。由于前面（第 2 行）已经验证过查询的选择条件可以下推穿过 gmap 的投影，这些连接操作数都是 psj 查询。算法尝试把它们的连接也表示成一个 psj 查询：扫描这些连接操作数（第 5 行），只要某一对操作数的自然连接等价于它们的加连接（第 6 行），就用一个新的连接操作数替换这一对，新操作数仍然是 psj 查询（第 7 行）。这样，连接操作数的集合不断缩小。到某一时刻，集合无法再缩小了：要么只剩下一个 psj 查询，要么再也找不到满足等价检验（第 6 行）的一对。在前一种情况下，剩下的 psj 查询在做完最后一步投影（ π_{Q_p} ）之后与最初的连接表达式 (2) 等价，并且可以在语法上检验它与逻辑查询是否等价（第 8 行）。在后一种情况下，第 4 行选出的子集被舍弃。算法 1 满足如下性质。

Proposition 1 *Given a psj-query Q and a set of psj-queries $\mathcal{G}$, for any subset $\{G_1, \dots, G_n\} \subseteq \mathcal{G}$ generated by Algorithm 1, the following holds:*

命题 1 给定一个 psj 查询 Q 和一个 psj 查询集合 $\mathcal{G}$ ，对于算法 1 生成的任一子集 $\{G_1, \dots, G_n\} \subseteq \mathcal{G}$ ，下式成立：

$$Q \equiv \pi_{Q_p} \sigma_{Q_s} (\pi_{A(Q_r)} G_1 \bowtie \dots \bowtie \pi_{A(Q_r)} G_n)$$

Because there are exponentially many subsets of $\mathcal{H}$, the whole algorithm runs in exponential time. However, checking if a given subset of gmaps can form a solution (lines 5-8) takes polynomial time. In the next section, we show how we can run the algorithm

in conjunction with a conventional optimizer to avoid enumerating all subsets.

由于 $\mathcal{H}$ 的子集有指数多个，整个算法的运行时间是指数级的。不过，检查给定的 gmap 子集能否构成一个解（第 5-8 行）只需多项式时间。下一节将说明如何把该算法与传统优化器结合起来运行，从而避免枚举所有子集。

An example may help illustrate the algorithm. Consider a query Q that asks for all 500-level courses, the names and department id's of students attending them:

举个例子可能有助于理解这个算法。考虑一个查询 Q ，它要查询所有 500 级课程，以及选这些课程的学生姓名和所在系的 id：

```
def_query Q by select Student.name, Dept
  where Student attends Course and
        Student enrolled Dept and Course.level = 500.
```

The database consists of three gmaps: an index G_1 from the names of students to their departments, an index G_2 from the names of students to courses they attend and the levels of those courses, and an index G_3 from a course-level to courses at that level, together with the departments that supply students to them.

数据库由三个 gmap 组成：索引 G_1 ，从学生姓名映射到其所在系；索引 G_2 ，从学生姓名映射到其所选课程以及这些课程的级别；索引 G_3 ，从课程级别映射到该级别的课程，同时给出为这些课程输送学生的系。

```

def_gmap G1 as btree by
  given Student.name select Dept
  where Student enrolled Dept
def_gmap G2 as btree by
  given Student.name select Course, Course.level
  where Student attends Course
def_gmap G3 as btree by
  given Course.level select Dept, Course
  where Student attends Course and Student enrolled Dept.

```

All three gmaps are relevant to the query (they pass the tests of lines 1–3). For example G1 can provide values for two attributes needed by the query (Dept and Student.name) so it passes line 1, it trivially satisfies the constraint test of line 2, and it covers the relations enrolled and Student.name, so it passes the test on line 3.

这三个 gmap 都与查询相关（它们都通过了第 1–3 行的检验）。例如，G1 能提供查询所需的两个属性（Dept 和 Student.name）的值，因此通过第 1 行；它平凡地满足第 2 行的约束检验；它覆盖了关系 enrolled 和 Student.name，因此通过第 3 行的检验。

The algorithm considers subsets of the relevant gmaps G1, G2, G3. Consider, for example, the subset { G1, G2 }. The candidate solution corresponding to this combination is the natural join of G1 and G2 followed by a selection for Course.level = 500 followed by a projection. The loop of lines 6 and 7 will be executed once to check whether $G1 \bowtie G2 = G1 \oplus G2$. This test will fail unless Student.name functionally determines Student; otherwise two tuples that join on the Student.name need not join on their Student id as well. If Student.name functionally determines Student, then the join on the Student id is irrelevant: we can project out that attribute before performing the join, which implies that the add-join is equivalent

to the natural join (line 6). Eventually line 8 of Algorithm 1 will conclude that the candidate solution is a correct one.

算法考察相关 gmap G_1 、 G_2 、 G_3 的各个子集。以子集 $\{G_1, G_2\}$ 为例。与这一组合对应的候选解是：先对 G_1 和 G_2 做自然连接，再做选择 $\text{Course.level} = 500$ ，最后做投影。第 6、7 行的循环会执行一次，检查 $G_1 \bowtie G_2 \equiv G_1 \oplus G_2$ 是否成立。除非 Student.name 函数决定 Student ，否则这个检验就会失败，因为在 Student.name 上能连接的两个元组，未必在 Student 的 id 上也能连接。如果 Student.name 函数决定 Student ，那么在 Student id 上的连接就无关紧要了：我们可以在连接之前把这个属性投影掉，这意味着加连接等价于自然连接（第 6 行）。最终，算法 1 的第 8 行会判定这个候选解是正确的。

Following the same process, the algorithm would reject the subsets $\{G_1, G_3\}$ and $\{G_2, G_3\}$, because the necessary multivalued dependencies do not hold. However, the combination $\{G_1, G_2, G_3\}$ is a correct solution. During the course of the loop of lines 6 and 7, the algorithm will test all pairs of gmaps in this subset to check whether or not their add-join is equivalent to their natural join. As we saw, all the pairs will fail except $G_1 \oplus G_2$. In the next iteration, the pair $(G_1 \oplus G_2, G_3)$ is considered and it is confirmed that its add-join is equivalent to its natural join.

按照同样的过程，算法会舍弃子集 $\{G_1, G_3\}$ 和 $\{G_2, G_3\}$ ，因为所需的多值依赖不成立。但组合 $\{G_1, G_2, G_3\}$ 是一个正确的解。在执行第 6、7 行循环的过程中，算法会检验该子集中所有的 gmap 对，看它们的加连接是否等价于自然连接。如前所见，除 $G_1 \oplus G_2$ 之外，所有的对都不满足。在下一轮迭代中，考察的是 $(G_1 \oplus G_2, G_3)$ 这一对，并确认它的加连接等价于自然连接。

Interestingly, the solution using all three gmaps is likely to be more efficient than the one that uses only G1 and G2, because the index on `Course.level` in G3 will accelerate the selection in the query. The next section shows how a gmap-aware optimizer identifies and prunes the inferior plan.

有意思的是，使用全部三个 gmap 的解很可能比只用 G1 和 G2 的解更高效，因为 G3 中 `Course.level` 上的索引能加速查询中的选择。下一节将说明一个能识别 gmap 的优化器如何识别并剪掉较差的计划。

4 Integration with a Query Optimizer

4 与查询优化器的集成

The presentation of Algorithm 1 emphasizes clarity at the expense of efficiency. It implies that all subsets of the gmaps are enumerated in random order and each is tested to see if it provides a solution to the equation. All subsets that pass the test are feasible plans. The version of the algorithm that is actually implemented by our system is considerably more sophisticated. It is integrated with a conventional dynamic-programming query optimizer [21], which controls the order in which subsets are evaluated and uses cost information and intermediate results to prune the search space.

算法 1 的表述重在清晰，而牺牲了效率。按照它的写法，gmap 的所有子集以任意顺序被枚举出来，逐个检验它是否是方程的一个解；所有通过检验的子集都是可行的计划。我们系统中实际实现的算法版本要复杂精巧得多。它与一个传统的基于动态规划的查询优化器 [21] 集成在一起，由优化器控制各子集的求值顺序，并利用代价信息和中间结果来剪枝搜索空间。

A conventional dynamic-programming optimizer iteratively finds optimal access plans for increasingly larger parts of a query. We follow these steps in more

detail, showing at each step what needs to be changed for a gmap-equipped database (Table 1). We then identify the pieces of Algorithm 1 that correspond to these changes. In what follows, for simplicity, we avoid any discussion of “interesting orders” [21]. We also use the term *complete solution* to refer to a gmap access plan (i.e., a specific sequence of joins, together with the method used for each join) that is equivalent to the logical query, and *partial solution* for a gmap access plan that could potentially be enhanced to become a complete solution. A partial solution does not necessarily have to be a psj-query; it may be that no reordering of its joins makes them equivalent to add-joins.

传统的动态规划优化器以迭代的方式，为查询中越来越大的部分找出最优访问计划。我们更细致地跟踪这些步骤，逐步说明对于配备了 gmap 的数据库需要做哪些改动（表 1），然后指出算法 1 中与这些改动相对应的部分。下文为简单起见，不讨论“有意义的顺序”（interesting orders）[21]。我们用“完整解”指与逻辑查询等价的 gmap 访问计划（即一个具体的连接序列，以及每个连接所用的方法），用“部分解”指有可能被扩充为完整解的 gmap 访问计划。部分解不一定是 psj 查询；有可能无论怎样调整其连接的次序，都无法使这些连接等价于加连接。

Like a conventional optimizer, the gmap optimizer only attempts to join a partial solution with gmaps that share projected attributes with it, thus avoiding Cartesian products. Each step in the gmap optimizer corresponds to part of Algorithm 1. Step (a1) of the first iteration corresponds to lines 1-3 of Algorithm 1; it finds all gmaps that are relevant to the query. The remaining steps of all iterations represent the rest of the algorithm. Moving from iteration to iteration and step (c) of each iteration corresponds to a specific implementation of line 4, where all subsets of relevant

gmaps that are not pruned on the way are regularly explored in increasing size. After the first iteration, step (a1) forms the joins of these subsets (solutions) and step (a2) corresponds to lines 6-8, where these solutions are examined for completeness. Step (a2) can be implemented incrementally taking into account the results of earlier iterations on smaller partial solutions.

与传统优化器一样，gmap 优化器只尝试把部分解与那些和它有共同投影属性的 gmap 相连接，从而避免笛卡儿积。gmap 优化器中的每一步都对应算法 1 的某一部分。第一轮迭代的步骤 (a1) 对应算法 1 的第 1-3 行，它找出所有与查询相关的 gmap；所有迭代中的其余步骤则对应算法的其余部分。从一轮迭代推进到下一轮，再加上每轮迭代中的步骤 (c)，相当于第 4 行的一种具体实现：所有未在中途被剪掉的相关 gmap 子集，按规模从小到大依次被考察。第一轮迭代之后，步骤 (a1) 构造这些子集（解）的连接，步骤 (a2) 对应第 6-8 行，在这里检查这些解是否完整。步骤 (a2) 可以增量地实现，利用之前各轮迭代在较小部分解上得到的结果。

Note that step (b) of each iteration has no counterpart in the translation algorithm because it deals only with pruning the search space and not with translation. Implementing this step is not straightforward because it involves not only the cost but also the contribution of solutions to the query. Contributions of partial solutions can be compared on the basis of their pieces that correspond to psj-queries and the set of attributes in their result. When each piece of a partial solution has subsets of the relations and projected attributes of a piece of another partial solution, then the former contributes less and can therefore be removed from further consideration if it also has a higher cost. Query signatures, an encoding of the names of all the relations used by the query, can be used to perform these comparisons efficiently [8].

注意，每轮迭代中的步骤 (b) 在翻译算法中没有对应部分，因为它只负责剪枝搜索空间，与翻译无关。实现这一步并不简单，因为它不仅要考虑解的代价，还要考虑解对查询的贡献。部分解的贡献可以根据两方面来比较：它们中对应于 psj 查询的各个片段，以及它们结果中的属性集合。如果一个部分解的每个片段所含的关系和投影属性，都是另一个部分解某个片段的关系和投影属性的子集，那么前者的贡献更小；如果它的代价还更高，就可以不再考虑它。查询签名是对查询所用全部关系名的一种编码，利用它可以高效地完成这些比较 [8]。

It is interesting to see how the new algorithm behaves when it is given a set of gmaps that represents a traditional relational physical schema. Assume for example that one gmap is a file containing the extent of the Faculty relation with all associated attributes,

看看新算法在给定一组表示传统关系物理模式的 gmap 时如何表现，是很有意思的。例如，假设一个 gmap 是一个文件，存放 Faculty 关系的外延及其所有相关属性，

```
def_gmap faculty_relation as heap by
  given Faculty select Faculty.name, Faculty.area, Dept
  where Faculty works_in Dept
```

while another gmap is a secondary index on the Faculty.area field,

而另一个 gmap 是 Faculty.area 字段上的二级索引，

```
def_gmap faculty_index_on_area as btree by
  given Faculty.area select Faculty.
```

Assume that the logical query requests the names of all faculty in the database area. During the first iteration both gmaps are considered. Scanning the relation extent would be far more expensive than accessing the index, but the two solutions are not comparable. Since

the index simply returns `Faculty` ids, it is not adequate to answer the query, while the extent is. During the second iteration, the index (the only partial solution left) is considered for a join with the `Faculty` extent. The join would be less expensive than scanning the `Faculty` extent while both plans are equivalent to the logical query. Thus, the solution found during the first iteration is eliminated in the second. At that point, there is no partial solution left and the algorithm ends. This example demonstrates that access plans that are pruned in the conventional optimizer are also pruned in its enhanced version. However, since an access plan considered at iteration n in the old version may combine more than n gmaps, it may be considered at a later iteration in the new version, thus delaying potential prunings. In general, we expect the performance of the modified optimizer to be similar to the performance of the original one. Our experience obtained by using the optimizer for the examples shown in Section 8 supports the prediction.

假设逻辑查询要求数据库方向所有教师的姓名。在第一轮迭代中，两个 gmap 都会被考虑。扫描关系外延要比访问索引昂贵得多，但这两个解不可比较：索引只返回 `Faculty` 的 id，不足以回答查询，而外延可以。在第二轮迭代中，索引（剩下的唯一部分解）被拿来与 `Faculty` 外延做连接。这个连接比扫描 `Faculty` 外延代价更低，而且两个计划都与逻辑查询等价。因此，第一轮找到的解在第二轮被淘汰。此时已没有剩余的部分解，算法结束。这个例子表明，在传统优化器中会被剪掉的访问计划，在其增强版本中同样会被剪掉。但由于旧版本在第 n 轮迭代中考虑的访问计划可能组合了多于 n 个 gmap，在新版本中它可能要更晚的迭代才被考虑，从而推迟了可能的剪枝。总的来说，我们预计修改后的优化器的性能与原来的相近。我们用该优化器处理第 8 节中那些例子所获得的经验，印证了这一预测。

Table 1: Step by step comparison of a conventional optimizer vs. one designed for a gmap-equipped database

Conventional optimizer	Gmap optimizer
Iteration 1 For each query relation: a) Find all possible access paths. b) Compare their cost and keep the least expensive. c) If the query involves only one relation, stop.	Iteration 1 a1) Find all gmaps that are relevant to the query. a2) Distinguish between partial and complete solutions among them. b) Compare all gmaps among themselves. If one has neither greater contribution to the query than another nor a lower cost, prune it. c) If there are no partial solutions, stop.
Iteration 2 For each query join: a) Consider joining the relevant access paths found in the previous iteration using all possible join methods. b) Compare the cost of the resulting join plans and keep the least expensive. c) If the query involves only two relations, stop.	Iteration 2 a1) Consider joining all partial solutions found in the previous iteration with another gmap using all possible join methods. a2) Distinguish partial and complete solutions among resulting joins. b) Compare all generated solutions among themselves and to any earlier solution. If any single gmap or gmap combination has neither a greater contribution to the query than another nor a lower cost, prune it. c) If there are no partial solutions, stop.
Iteration 3	Iteration 3

表 1: 传统优化器与为配备 gmap 的数据库设计的优化器的逐步比较
【传统优化器】

第 1 轮：对每个查询关系——

- a) 找出所有可能的访问路径。
- b) 比较它们的代价，保留代价最低的。
- c) 如果查询只涉及一个关系，停止。

第 2 轮：对每个查询连接——

- a) 考虑把上一轮找到的相关访问路径，用所有可能的连接方法连接起来。
- b) 比较所得连接计划的代价，保留代价最低的。
- c) 如果查询只涉及两个关系，停止。

第 3 轮：……

【gmap 优化器】

第 1 轮：

- a1) 找出所有与查询相关的 gmap。
- a2) 区分其中的部分解和完整解。
- b) 在所有 gmap 之间两两比较。如果某个 gmap 对查询的贡献不比另一个大，代价也不比它低，就把它剪掉。
- c) 如果没有部分解了，停止。

第 2 轮：

- a1) 考虑把上一轮找到的所有部分解与另一个 gmap，用所有可能的连接方法连接起

来。

a2) 在得到的连接中区分部分分解和完整解。

b) 在所有新生成的解之间，以及它们与之前任何一个解之间进行比较。如果某个 gmap 或 gmap 组合对查询的贡献不比另一个大，代价也不比它低，就把它剪掉。

c) 如果没有部分分解了，停止。

第 3 轮：……

5 Update propagation

5 更新传播

Relational systems mitigate dependencies between the logical and the physical schema through the use of stored queries called *views*, and users express their queries in terms of the views. With this approach, the logical schema becomes a (relational) function of the physical schema. View updates, however, are difficult or impossible to support. The usual solution is to require updates to be expressed in terms of the underlying schema.

关系系统借助称为视图的存储查询来缓解逻辑模式与物理模式之间的依赖，用户基于视图来表达查询。在这种做法下，逻辑模式成了物理模式的一个（关系）函数。然而，视图更新很难支持，甚至无法支持。通常的解决办法是要求用底层模式来表达更新。

Our approach is the inverse. We define the physical structures as functions of the logical schema. Although query translation is more complicated, we have shown above that it is still possible, and it can be integrated with the optimization stage of a conventional system adding little overhead to the preparation of query plans and no overhead to the execution of those plans. Updates, however, are much simpler. Translating them into the physical schema turns into the

materialized view maintenance problem, which accepts simple solutions.

我们的做法正好相反：我们把物理结构定义为逻辑模式的函数。虽然查询翻译变得更复杂，但前面已经表明它仍然可行，而且可以集成到传统系统的优化阶段中，只给查询计划的准备过程增加很少的开销，而对计划的执行则毫无额外开销。而更新则要简单得多。把更新翻译到物理模式上，就变成了物化视图维护问题，这个问题有简单的解法。

As discussed elsewhere [3], propagating updates into materialized views requires the execution of queries over the base relations and the inserted or deleted tuples. However, here we do not necessarily have the base relations stored, and the actual data are replicated in many places. In this section, we illustrate how the query translation algorithm described above can be adapted to translate an update request over the logical schema into the corresponding physical plan. Our algorithm produces optimal update plans, using existing gmaps to accelerate update propagation where possible.

正如 [3] 中讨论的，把更新传播到物化视图，需要在基本关系以及被插入或删除的元组上执行查询。然而，在这里基本关系不一定被存储下来，而实际数据又被复制在许多地方。本节说明如何改造前面介绍的查询翻译算法，把针对逻辑模式的更新请求翻译成相应的物理计划。我们的算法生成最优的更新计划，并尽可能利用已有的 gmap 来加速更新传播。

5.1 Specifying Updates

5.1 更新的指定

Insertions are specified by supplying a query (the *update query*) and a set of tuples to be inserted (the *update data*), corresponding to the target attributes of

the query. The database must be updated in such a way that the change in the results of the query between the original and updated database is precisely the set of tuples in the update data. Deletions are defined similarly, with the roles of “original” and “updated” database reversed. Note the difference from the query used when specifying updates in SQL-like languages, in which the query is used to generate the update tuples. The query here describes only the “schema” of the tuples. Since the update data can be the result of another query, no generality is lost.

插入操作通过给出一个查询（更新查询）和一组待插入的元组（更新数据）来指定，这些元组对应于该查询的目标属性。数据库必须这样更新：查询在原数据库和更新后数据库上的结果之差，恰好就是更新数据中的元组集合。删除的定义与此类似，只是把“原”数据库和“更新后”数据库的角色对调。注意，这与类 SQL 语言中指定更新时所用的查询不同：在那里，查询用来生成要更新的元组；而这里的查询只描述元组的“模式”。由于更新数据本身可以是另一个查询的结果，这样做并不损失通用性。

For example, students can become enrolled in courses by supplying a set of *(StudentId, CourseId)* pairs and the update query

例如，要让学生选修课程，可以给出一组 *(StudentId, CourseId)* 对以及如下更新查询：

```
def_query enroll_student as
  select Student, Course where Student attends Course.
```

Allowing arbitrary queries to be used in update gmaps would re-introduce all the problems of updating through views. Therefore, we disallow projections and explicit selections from update queries and impose a few other restrictions described in more detail elsewhere [25]. Although the semantics of the update

query depends only on the logical schema, its validity may depend on the choice of gmaps used to define the physical schema. For example, the physical schema must have sufficient “information capacity” to hold the inserted data [17]. These issues are not addressed in this paper.

如果允许在更新 gmap 中使用任意查询，就会把通过视图进行更新的所有问题重新带回来。因此，我们不允许更新查询中出现投影和显式选择，并施加了其他一些限制，详见 [25]。虽然更新查询的语义只取决于逻辑模式，但它是否合法可能取决于定义物理模式时选用了哪些 gmap。例如，物理模式必须有足够的“信息容量”来容纳插入的数据 [17]。本文不讨论这些问题。

5.2 The Algorithm

5.2 算法

Given an update query U , a set ΔU of tuples, and a gmap G in the physical schema, we need to find the change ΔG in the value of G corresponding to the change ΔU in the value of U . Suppose we find a collection of psj-queries $H_1, \dots, H_m$ that are invariant under changes to U such that

给定一个更新查询 U 、一个元组集合 ΔU 以及物理模式中的一个 gmap G ，我们需要求出 U 的值变化 ΔU 时， G 的值相应的变化 ΔG 。假设我们找到了一组 psj 查询 $H_1, \dots, H_m$ ，它们在 U 变化时保持不变，并且满足

$$G = \pi_{G_p} \sigma_{G_s}(U \bowtie H_1 \bowtie \dots \bowtie H_m).$$

Let G_{prev} be the value of G before the update and G_{new} the value after. Then

设 G_{prev} 为更新前 G 的值， G_{new} 为更新后的值。那么

$$\begin{aligned}
G_{new} &= \pi_{G_p} \sigma_{G_s}((U + \Delta U) \bowtie H_1 \bowtie \dots \bowtie H_m) \\
&= \pi_{G_p} \sigma_{G_s}(\Delta U \bowtie H_1 \bowtie \dots \bowtie H_m) + G_{prev}
\end{aligned}$$

or equivalently

或者等价地,

$$\Delta G = \pi_{G_p} \sigma_{G_s}(\Delta U \bowtie H_1 \bowtie \dots \bowtie H_m). \quad (3)$$

In other words, the updates to G can be found by evaluating the right-hand side of (3). As shown elsewhere [25], we can use Algorithm 1 to find $H_1, \dots, H_m$. Here, we illustrate the algorithm with an example.

换句话说, 只要计算 (3) 式的右边, 就能求出 G 的更新。如 [25] 所示, 可以用算法 1 来求 $H_1, \dots, H_m$ 。这里我们用一个例子来说明该算法。

Consider the update query `enroll_student` presented earlier

考虑前面给出的更新查询 `enroll_student`:

```
def_query enroll_student as select Student, Course
where Student attends Course.
```

Assume that our database consists of two gmaps, one that maps faculty members to the courses they teach,

假设数据库由两个 gmap 组成, 其中一个把教师映射到他们所教的课程,

```
def_gmap FC as btree by given Faculty select Course
where Faculty teaches Course,
```

and one that records the students and teacher of each course,

另一个记录每门课程的学生和授课教师,

```
def_gmap CFS as btree by
  given Course select Faculty, Student
  where Faculty teaches Course and
        Student attends Course.
```

To propagate the update to the database, we consider each database gmap separately. The gmap FC is not affected by the update since it has no common relations with the update query. The updates to gmap CFS depend both on the update data and on the existing contents of FC. We need to enhance the (*CourseId*, *StudentId*) pairs in the update data with the faculty members who teach the courses before they are added to CFS. The algorithm constructs the tuples to be inserted by considering the part of the gmap that is not affected by the update, i.e., the query

为了把更新传播到数据库，我们逐个考虑数据库中的 gmap。gmap FC 不受这次更新影响，因为它与更新查询没有共同的关系。gmap CFS 的更新既取决于更新数据，也取决于 FC 现有的内容：在把更新数据中的 (*CourseId*, *StudentId*) 对加入 CFS 之前，需要为它们补上讲授这些课程的教师。算法通过考察 gmap 中不受更新影响的那部分来构造待插入的元组，也就是查询

```
def_query Q by select Course, Faculty
  where Faculty teaches Course,
```

and trying to find a translation for it. Q can definitely be answered by using gmap FC, and thus the tuples to be inserted into CFS are found by joining the update gmap `enroll_student` and the gmap FC. Gmap CFS may not be used as the source of the needed information because CFS will not contain (*CourseId*, *FacultyId*) pairs for courses that do not yet have any students. Line 2 of Algorithm 1 tests whether CFS covers the relation `teaches`. As long as there is no inclusion dependency from `teaches` to `attends`, the CFS gmap

will be rejected.

并尝试为它找到一个翻译。Q 显然可以用 gmap FC 来回答，因此，待插入 CFS 的元组可以通过把更新 gmap enroll_student 与 gmap FC 做连接得到。gmap CFS 不能作为所需信息的来源，因为对于还没有任何学生选修的课程，CFS 中不会有相应的 (CourseId, FacultyId) 对。算法 1 的第 2 行检验 CFS 是否覆盖关系 teaches。只要不存在从 teaches 到 attends 的包含依赖，gmap CFS 就会被排除。

Equation (3) can be used for deletions as well as insertions, provided each gmap whose target does not functionally determine its other attributes maintains its data as a multiset (that is, it records duplicate insertions or a count of them).

方程 (3) 既可用于插入，也可用于删除，前提是：如果某个 gmap 的目标属性不能函数决定它的其他属性，那么该 gmap 要以多重集的形式保存数据（即记录重复插入的元组，或记录重复的次数）。

6 Applications

6 应用

In Section 2, we demonstrated how gmaps subsume the facilities of primary and secondary storage structures in conventional database systems. In this section, we outline a variety of other applications that use the integrated query translation-optimization engine for queries and updates.

在第 2 节中，我们展示了 gmap 如何涵盖传统数据库系统中主存储结构和二级存储结构的功能。本节概述其他多种应用，它们都利用了这套集查询翻译与优化于一体的引擎来处理查询和更新。

6.1 Database Loading

6.1 数据库装载

Existing DBMSs often provide special features to support bulk loading of data. These facilities tend to be *ad hoc* and impose restrictions on the format of the imported data. The user must manually translate all imported data into files that match the primary storage structures and then load each one individually. If the imported data can be described as a psj-query on the logical schema, initial loading can be viewed as a special case of updates. The imported data files can be viewed as inefficient gmap structures. The “real” gmaps used to store the data permanently are loaded by running their queries against the imported files.

现有的 DBMS 通常提供专门的功能来支持数据的批量装载。这些功能往往是临时拼凑的，而且对导入数据的格式有诸多限制。用户必须手工把所有导入数据转换成与主存储结构相匹配的文件，然后逐个装载。如果导入的数据可以描述为逻辑模式上的一个 psj 查询，那么初始装载就可以看作更新的一个特例：导入的数据文件可以看作低效的 gmap 结构，而用于永久存储数据的那些“真正的” gmap，则通过在导入文件上运行它们的查询来装载。

6.2 Accelerating Complex Structure Updates

6.2 加速复杂结构的更新

Although path indices are useful for accelerating common queries, they are expensive to maintain. Previous proposals for the maintenance of complex access paths suggest using *ad hoc* techniques to accelerate expensive updates, such as internal links between instances in nested indices and field replication [23, 2] or links from instances to their collections [16, 18]. We can achieve the same effect by using simple gmaps along

the paths that need to be traversed during the update propagation. Gmaps placed at points where expensive joins are performed act like join accelerators in the same way that internal links accelerate joins. These gmaps can accelerate not only updates to the structure concerned, but any other query or update to which they apply. If usage patterns change so that the cost of maintaining the accelerators exceeds their benefit, we can simply remove them. In the hardwired approach, we will always be stuck with the same machinery. Furthermore, our approach allows the user to place join accelerators at specific points. It is much harder to achieve this flexibility with the hardwired approach.

路径索引虽然有助于加速常见查询，但维护代价很高。以往关于复杂访问路径维护的方案，建议用一些专门的技术来加速代价高的更新，例如在嵌套索引和字段复制中使用实例之间的内部链接 [23, 2]，或者从实例指向其所属集合的链接 [16, 18]。我们可以达到同样的效果：沿着更新传播时需要遍历的路径放置一些简单的 gmap。放在需要执行高代价连接之处的 gmap 起到连接加速器的作用，就像内部链接加速连接一样。这些 gmap 不仅能加速相关结构的更新，还能加速它们适用的任何其他查询或更新。如果使用模式发生变化，以致维护这些加速器的代价超过了收益，只需把它们删掉即可。而在硬连线的做法中，我们永远只能用同一套固定机制。此外，我们的做法允许用户在指定的位置放置连接加速器，而硬连线的做法很难实现这种灵活性。

6.3 Other applications

6.3 其他应用

Using update queries to describe the schema of the modified data facilitates the handling of complex objects. A complex object insertion can be described as an update gmap whose query describes the complex object schema. If the inserted object consists of

several tuples of data, the tuples are not inserted one at a time. Instead, the query plan treats the update gmap like any other physical data container and performs bulk operations. The result of the query plan execution is a set of tuples to be inserted into each physical storage structure. Any available bulk-loading interfaces to these structures can be exploited.

用更新查询来描述被修改数据的模式，便于处理复杂对象。复杂对象的插入可以描述为一个更新 gmap，其查询描述了该复杂对象的模式。如果被插入的对象由若干个数据元组组成，这些元组并不是逐个插入的；查询计划会像对待其他任何物理数据容器一样对待这个更新 gmap，执行批量操作。查询计划的执行结果是要插入到每个物理存储结构中的一组元组。这些结构上现有的任何批量装载接口都可以加以利用。

In Section 6.1, we discussed how to create a thin “veneer” over imported text files to make them behave like gmaps. This idea can be extended to support heterogeneous storage organizations by hiding their differences under the gmap abstraction. As long as the data contents of all these distinct sources can be described by psj-queries over a single logical schema, the query optimizer can translate logical queries into access plans over them. This strategy is similar to the ADMS handling of database interoperability [19, 20].

在 6.1 节中，我们讨论了如何在导入的文本文件之上加一层薄薄的“外壳”，使它们表现得像 gmap 一样。这个思路可以推广：把各种异构存储组织的差异隐藏在 gmap 抽象之下，从而支持异构的存储组织。只要所有这些不同数据源的数据内容都能用同一个逻辑模式上的 psj 查询来描述，查询优化器就能把逻辑查询翻译成在它们之上的访问计划。这一策略与 ADMS 处理数据库互操作的方式类似 [19, 20]。

Another application of gmaps is to support cached data in transient main-memory data structures such as arrays or hash tables. If the contents of the structure can be described by a psj-query on the logical schema, it can be treated like a gmap.

gmap 的另一个应用，是支持缓存在临时性主存数据结构（如数组或哈希表）中的数据。只要这种结构的内容可以用逻辑模式上的 psj 查询来描述，它就可以当作 gmap 来处理。

7 Implementation

7 实现

To verify the applicability and practicality of our algorithms and obtain a feeling for their performance, we built a prototype implementation of our system on top of SHORE [4]. SHORE is an object-oriented database system under construction at the University of Wisconsin. Logical schema definitions are parsed and stored in a logical-schema catalog. Physical storage structures are created from gmap definitions. The parsed gmap query is stored persistently in a second physical-schema catalog. The data organization, keys, and record format are also determined by the gmap definition. The gmap is created and populated by processing an update request.

为了验证我们算法的适用性和实用性，并对其性能有个直观的认识，我们在 SHORE [4] 之上构建了系统的一个原型实现。SHORE 是威斯康星大学正在开发的一个面向对象数据库系统。逻辑模式定义经过解析后存入逻辑模式目录。物理存储结构根据 gmap 定义创建。解析后的 gmap 查询被持久地存储在另一个目录，即物理模式目录中。数据组织方式、键和记录格式也由 gmap 定义确定。gmap 的创建和填充通过处理一个更新请求来完成。

We built a query processor using the algorithms in Sections 3 and 4. For all the examples presented in this paper, query translation added only a negligible overhead to the overall query cost. The query processor also contains hooks to support the update processor, as outlined in Section 5. The update processor accepts three lists of gmaps: update gmaps, target gmaps to be updated, and database gmaps that may be used to supply data for the updates. For simple updates, the first list contains just one gmap and the target and database lists each contain all of the gmaps in the physical-schema catalog. Other combinations of arguments support other applications described in Section 6.

我们使用第 3、4 节中的算法构建了一个查询处理器。对于本文给出的所有例子，查询翻译给整体查询代价增加的开销都可以忽略不计。查询处理器中还有一些钩子，用来支持第 5 节所述的更新处理器。更新处理器接受三个 gmap 列表：更新 gmap；待更新的目标 gmap；以及可用来为更新提供数据的数据库 gmap。对于简单的更新，第一个列表只含一个 gmap，目标列表和数据库列表都包含物理模式目录中的全部 gmap。参数的其他组合则用来支持第 6 节描述的其他应用。

We designed a simple common interface for storage structures, including operations to store and retrieve data and to make cost inquiries. We implemented this interface on top of existing SHORE facilities (B^+ -trees and heaps) as well as Unix files (for importing and exporting data) and main-memory structures (for update gmaps). To support the use of the algorithm in Section 5 for deletions, we also modified the SHORE B^+ -tree facility to maintain a count of duplicate insertions rather than rejecting them.

我们为存储结构设计了一个简单的公共接口，包括存储数据、检索数据

以及查询代价的操作。我们在 SHORE 现有的设施（B+ 树和堆）之上实现了这个接口，也在 Unix 文件（用于导入和导出数据）和主存结构（用于更新 gmap）之上实现了它。为了支持把第 5 节的算法用于删除操作，我们还修改了 SHORE 的 B+ 树设施，使其对重复插入不再拒绝，而是维护一个重复计数。

8 A Performance Demonstration

8 性能演示

In this section, we describe experiments with a test database that illustrate that for a plausible mix of queries and updates, our techniques can provide better performance than either relational or object-oriented databases. As we observed in Section 2, gmaps can be used to describe the relations and the primary and secondary indices of relational databases, as well as the class extents, object sets, and path indices of object-oriented databases. Thus, we are able to use our system to simulate two “conventional” configurations, one based on a normalized relational design and one following a typical object-oriented database design. All of our results are reported in terms of counts of I/O operations, since absolute performance in “real” databases would be affected by a variety of implementation-dependent features that are beyond our control.

本节介绍在一个测试数据库上所做的实验。实验表明，对于一组合理的查询与更新组合，我们的技术能提供比关系数据库和面向对象数据库都更好的性能。正如第 2 节所述，gmap 既可以描述关系数据库中的关系以及主索引和二级索引，也可以描述面向对象数据库中的类外延、对象集合和路径索引。因此，我们可以用自己的系统来模拟两种“传统”配置：一种基于规范化的关系设计，另一种采用典型的面向对象数据库设计。我们的所有结果都以 I/O 操作次数来报告，因为在“真实”数据库中，绝对性能会受到各种依赖于具体实现、超出我们控制范围的因素影响。

In the experiment, we used an extended version of the university database presented earlier. We populated one department with actual data describing the Computer Sciences Department at the University of Wisconsin—Madison and generated synthetic data for 99 more departments to create a database of reasonable size. While the actual database used for the experiment includes additional fields, for simplicity we only discuss the logical schema as presented in Figure 1. A few interesting parameters of the data are presented in Table 2.

实验中，我们使用了前面介绍的大学数据库的一个扩展版本。我们用威斯康星大学麦迪逊分校计算机科学系的真实数据填充了一个系，又为另外 99 个系生成了合成数据，从而得到一个规模合理的数据库。虽然实验实际使用的数据库还包含一些其他字段，但为简单起见，我们只讨论图 1 所示的逻辑模式。表 2 列出了数据的几个值得关注的参数。

Table 2: Parameters of the database

	Faculty	Students	Courses	TAs	Depts
Instances	5000	50000	10000	2000	100
KB/inst	1	0.8	1	0.85	3

表 2：数据库参数（Instances：实例数；KB/inst：每个实例的大小，单位 KB）

8.1 The Workload

8.1 负载

The workload contains multiple runs of eight queries and five updates. The actual number of runs was determined by various factors. We tried to maintain a balance between expensive queries and simple ones, hold the update load at about a third of the total load for most of the configurations, and make the relative frequencies as realistic for a university environ-

ment as possible. Tables 3 and 4 briefly describe each query and update. For example, the workload contains three runs of query Q6. The last two columns contain the total cost attributed to all runs of query Q6 when it is processed on two different configurations of the database: a relational one (Section 8.2) and an object-oriented one (Section 8.3). Note that each update inserts a single pair of values. For example, when a student adds a course (U3), the inserted pair would be (*student id*, *course id*).

负载包含八个查询和五个更新，每个都运行多次。具体的运行次数由多种因素决定。我们力求在代价高的查询和简单的查询之间保持平衡，对大多数配置让更新负载约占总负载的三分之一，并尽可能让各操作的相对频率符合大学环境的实际情况。表 3 和表 4 简要描述了每个查询和更新。例如，负载中包含查询 Q6 的三次运行。最后两列给出查询 Q6 的全部运行在数据库的两种不同配置上处理时的总代价：一种是关系配置（8.2 节），另一种是面向对象配置（8.3 节）。注意，每次更新只插入一对值。例如，当一名学生加选一门课程（U3）时，插入的就是 (student id, course id) 这一对。

Table 3: Queries in the workload

No	Mix	Given	Find	Rel	OO
Q1	10	faculty name	field, dept	30	30
Q2	10	faculty name	courses taught	30	30
Q3	10	student name	year, dept, advisor	50	50
Q4	10	TA name	support level, course	40	40
Q5	3	faculty area	students advised	57	57
Q6	3	student name	courses, teachers	39	42
Q7	2	course name	students attending	84	80
Q8	1	dept name	courses taught	64	59

表 3：负载中的查询（No：编号；Mix：运行次数；Given：给定；Find：查找；Rel：关系配置下的代价；OO：面向对象配置下的代价）

Table 4: Updates in the workload

No	Mix	Update Description	Rel	OO
U1	1	Faculty teaches a new course	5	7
U2	2	Faculty starts advising a student	10	14
U3	20	Student adds a course	160	120
U4	4	TA is assigned to a course	20	20
U5	6	Student enrolls in a department	24	24

表 4: 负载中的更新 (U1 教师开设一门新课; U2 教师开始指导一名学生; U3 学生加选一门课程; U4 为助教分配一门课程; U5 学生注册到某个系)

To obtain I/O counts, we instantiated queries and updates using random values for the input parameters and the inserted pairs. Then, we used the actual plans obtained by our system to perform the query or update and recorded the size of the result as well as the sizes of all intermediate results used in joins. From these data, we calculated I/O counts using a simple model of the B⁺-tree and the heap storage structures based on the assumptions that the page size is 8KB and that 4MB are used for buffers. The analytical approach was chosen in favor of measuring actual response times, since the underlying SHORE system was still under development.

为了得到 I/O 次数, 我们用随机值来实例化查询和更新的输入参数以及插入的值对。然后, 用我们系统实际生成的计划执行查询或更新, 并记录结果的大小, 以及连接中用到的所有中间结果的大小。根据这些数据, 我们用 B+ 树和堆存储结构的一个简单模型计算出 I/O 次数, 假设页大小为 8KB, 缓冲区为 4MB。之所以选择这种分析方法而不是测量实际响应时间, 是因为底层的 SHORE 系统仍在开发中。

8.2 Relational Design

8.2 关系设计

The relational configuration follows a textbook translation of the logical schema into relations. We added secondary indices as needed, to improve joins and selections, and we clustered favorably the records in the heaps. The total database size for this configuration is 74MB. Running the full workload on that database costs 613 page I/Os, 64% caused by queries and 36% by updates. The exact contribution of all runs of each query and update in the total cost is shown in Tables 3 and 4.

关系配置按照教科书式的方法把逻辑模式转换为关系。我们根据需要添加了二级索引来改进连接和选择，并对堆中的记录做了有利的聚簇。这种配置下数据库的总大小为 74MB。在该数据库上运行完整负载的代价是 613 次页 I/O，其中 64% 来自查询，36% 来自更新。每个查询和更新的全部运行在总代价中所占的具体份额见表 3 和表 4。

8.3 Object-Oriented Design

8.3 面向对象设计

The physical schema for the object-oriented configuration includes one extent file for each internal domain in the logical schema. The relationship *attends* is stored both as part of student objects and as part of the course objects, i.e., students contain pointers to all courses they attend, and courses contain pointers to the students attending them. This duplication allows efficient execution of queries Q6 and Q7. Each other relationship is stored as part of the domain closest to the edge label in Figure 1. The total database size for this configuration is 75MB. Running the full workload on the database costs 583 page I/Os, 68% caused by queries and 32% by updates. The last column of Tables 3 and 4 show the costs for all runs of each query and update.

面向对象配置的物理模式为逻辑模式中的每个内部域设置一个外延文件。关系 attends 既作为学生对象的一部分存储，也作为课程对象的一部分存储，也就是说，学生对象包含指向其所选全部课程的指针，课程对象包含指向选修该课程的学生对象的指针。这种重复存储使查询 Q6 和 Q7 能够高效执行。其他每个关系都作为图 1 中离边标签最近的那个域的一部分来存储。这种配置下数据库的总大小为 75MB。在该数据库上运行完整负载的代价是 583 次页 I/O，其中 68% 来自查询，32% 来自更新。表 3 和表 4 的最后一列给出了每个查询和更新全部运行的代价。

8.4 Object-Oriented Design with Complex Access Paths

8.4 带复杂访问路径的面向对象设计

The previous two configurations do not use any complex access path such as path indices¹ or field replication. In this section, we consider the case of an object-oriented DBMS equipped with such access paths. For the given workload, field replication can be applied in three cases, as shown in Table 5. Each row of the table describes what is replicated, the additional space required in MB, the affected queries (no update is affected by these additions), and the savings that can be attributed to the replication for all runs of the affected query.

前面两种配置都没有使用路径索引¹或字段复制这类复杂访问路径。本节考虑配备了此类访问路径的面向对象 DBMS。对于给定的负载，字段复制可以在三种情况下应用，如表 5 所示。表中每一行说明复制的内容、所需的额外空间（MB）、受影响的查询（这些新增内容不影响任何更新），以及对受影响查询的全部运行而言，可归功于该复制的节省量。

Table 5: Conventional complex path techniques

Replicate	In	Space	No	Saved
"Faculty.work.in.name"	Faculty	0.2	Q1	10
"Student.enrolled.name"	Student	2	Q3	10
"TA.assists.name"	TA	0.1	Q4	20
Tot. saved				40

表 5: 传统的复杂路径技术 (Replicate: 复制内容; In: 存放于; Space: 空间; No: 受影响查询; Saved: 节省; Tot. saved: 总节省)

The rest of the physical schema is identical to the earlier object-oriented schema. The data replication adds 2MB of additional space bringing the total database size to 77MB. The new database offers a 7% performance gain over the previous one. It is interesting to note that the technique of field replication as originally described [11, 23] cannot be applied to any other field in our database because it is restricted to edges from classes to single-valued attributes. Similar restrictions also prohibit any useful application of path indices in our database. The restrictions are there for a reason: the implementation of these access paths and the task of propagating updates would be far more complex without them.

物理模式的其余部分与前面的面向对象模式完全相同。数据复制增加了 2MB 空间, 使数据库总大小达到 77MB。新数据库比之前的性能提升了 7%。值得注意的是, 按最初描述的字段复制技术 [11, 23] 无法应用到我们数据库中的其他任何字段, 因为它只限于从类指向单值属性的边。类似的限制也使得路径索引在我们的数据库中派不上任何用场。这些限制是有原因的: 如果没有它们, 这些访问路径的实现以及更新传播的工作都会复杂得多。

8.5 A Configuration with Gmaps

8.5 使用 gmap 的配置

In this section, we consider our approach, i.e., a database fully equipped with gmaps. Instead of designing a physical organization from scratch, we show how we can make incremental changes on the object-oriented physical schema to improve its performance.

本节考虑我们的方法，即一个全面配备了 gmap 的数据库。我们并不从头设计物理组织，而是说明如何对面向对象的物理模式做增量式的修改，以提升其性能。

First, we replicate attributes over paths that traverse relationships both in the inverse direction (from attribute to class), and in the forward direction (from class to attribute). Such a replication brings cost savings in queries Q3 and Q7. Then, we consider replicating attributes over paths that are not functional by associating multiple values with each instance of the root of the path. Such replication is very beneficial for our workload since it can eliminate the most expensive step of the medium and large queries Q5, Q7, and Q8. The exact cost savings and overheads that are caused by each additional access path are shown in Table 6.

首先，我们沿着路径复制属性，这些路径既包括反向（从属性到类）穿过关系的，也包括正向（从类到属性）穿过关系的。这种复制为查询 Q3 和 Q7 节省了代价。然后，我们考虑沿非函数型路径复制属性，即为路径根部的每个实例关联多个值。这种复制对我们的负载非常有利，因为它能消除中型和大型查询 Q5、Q7 和 Q8 中代价最高的步骤。每条新增访问路径带来的具体节省和开销见表 6。

¹We use the term path indices collectively for what Bertino and Kim [2] call path indices, nested indices, and multi-indices.

¹我们用“路径索引”一词统称 Bertino 和 Kim [2] 所说的路径索引、嵌套索引和多重索引。

Table 6: Applications of gmaps

Replicate	In	Space	No	Saved
"Student.advises ⁻¹ .name"	Student	2	Q3,U2	18
"Course.teaches ⁻¹ .name"	Course	0.4	Q6,U1	9
"Faculty.advises.name"	Faculty	0.6	Q5	51
"Course.attends.name"	Course	12	Q7	76
"Faculty.teaches.name"	faculty	0.4	Q8	50
Tot. saved				204

表 6: gmap 的应用

These modifications offer a significant total gain of 204 I/Os, or 35% over the cost of the initial object-oriented approach. If we add to these savings those of the previous section (with gmaps simulating field replication) the total gain climbs to 42%. It is interesting to note the low update overhead of all these replications. The reason is that we replicated attributes, like `student.name`, that are static, i.e., do not get updated. As a result, the only updates that are affected are those on the intermediate links of the path. The `attends` relationship, for example, is the most volatile relationship in the schema, so we expect that the slightest update overhead would overwhelm any savings gained. However, since the `attends` relationship is bi-directional (stored as an attribute of both the student and the course), both extents need to be updated anyway. Reading and writing one more attribute, `student.name`, does not add any overhead.

这些修改带来了显著的总收益：减少 204 次 I/O，即比最初的面向对象方案的代价降低 35%。如果再加上上一节的节省（用 gmap 模拟字段复制），总收益升至 42%。值得注意的是，所有这些复制带来的更新开销都很低。原因在于，我们复制的是像 `student.name` 这样的静态属性，即不会被更新的属性。因此，受影响的更新只有路径中间环节上的那些。例如，`attends` 是模式中变动最频繁的关系，因此我们本以为哪怕一点点更新开销都会抵消掉

所有节省。然而，由于 attends 关系是双向存储的（既作为学生的属性，也作为课程的属性），两个外延反正都要更新。多读写一个属性 student.name 并不会增加任何开销。

The space overhead is also low in all but one case: replicating the student names in each course more than doubles the size of the course extent. Whether or not the space-time trade-off is worthwhile depends on the application. The total increase in space usage is 15.4MB, bringing the total database size to 92.4MB. Table 7 compares the query cost, update cost, and disk usage of all four approaches. For the gmap configuration, we show the results both with and without replication of student names, always including the enhancements of the configuration with complex paths.

除一种情况外，空间开销也都很低：在每门课程中复制学生姓名，会使课程外延的大小增加一倍以上。这种空间与时间的权衡是否值得，取决于具体应用。空间使用总共增加了 15.4MB，使数据库总大小达到 92.4MB。表 7 比较了四种方案的查询代价、更新代价和磁盘占用。对于 gmap 配置，我们分别给出了复制学生姓名和不复制学生姓名时的结果，两者都包含了复杂路径配置中的那些增强。

Table 7: Summary costs

	Space (MB)	Query cost	Update cost	Tot. cost
Relational	74	394	219	613
Object-oriented	75	398	185	583
OO + extensions	77	358	185	543
Gmaps (1)	80.4	227	188	415
Gmaps (2)	92.4	151	188	339

表 7：代价汇总（Relational：关系；Object-oriented：面向对象；OO + extensions：面向对象 + 扩展；Gmaps (1)/(2)：不复制/复制学生姓名的 gmap 配置）

9 Related Work

9 相关工作

Most existing database systems do not provide true data independence, since every construct of the logical schema corresponds directly to a primary physical structure. For example, every relation in most relational systems, every class extent in extent-based OO systems (e.g., Orion [15] and Zoo [10]) and every collection in collection-based OO systems (e.g., GemStone [16], Extra/Excess [5], and ObjectStore [18]) is stored in a separate file. The main flexibility at the physical level comes from secondary access paths to these files.

大多数现有数据库系统并不提供真正的数据独立性，因为逻辑模式中的每个结构都直接对应一个主物理结构。例如，大多数关系系统中的每个关系、基于外延的面向对象系统（如 Orion [15] 和 Zoo [10]）中的每个类外延，以及基于集合的面向对象系统（如 GemStone [16]、Extra/Excess [5] 和 ObjectStore [18]）中的每个集合，都存放在单独的文件中。物理层的灵活性主要来自于这些文件上的二级访问路径。

Several extensions of both the primary physical structure and the secondary access paths have been recently proposed in the literature that allow storing together data from more than one logical construct. In Sections 2 and 8, we discussed path indices [2, 14, 16], join indices [26], and field replication [11, 23], noting their restrictions and comparing their performance to our scheme. Another approach to decomposing the database is hierarchical join indices [27], a generalization of join indices that allows one to build an index over identity surrogates that populate trees of the logical schema graph. Access Support Relations (ASR) offer a different generalization of join indices [12], which

allows the definition of indices over the instances of arbitrary chains of logical schema nodes. This scheme offers a higher degree of flexibility and allows the definition of indices that store both complete and partial instances of each chain. Except for the last feature, the contents of both hierarchical join indices and ASRs can be represented as psj-queries, and can thus be defined as a gmap. However, since gmap queries do not support unions, they cannot represent outer joins, and therefore cannot store incomplete instances of chains.

近来文献中提出了对主物理结构和二级访问路径的若干扩展，允许把来自多个逻辑结构的数据存放在一起。在第 2 节和第 8 节中，我们讨论了路径索引 [2, 14, 16]、连接索引 [26] 和字段复制 [11, 23]，指出了它们的限制，并把它们的性能与我们的方案做了比较。另一种分解数据库的方法是层次连接索引 [27]，它是连接索引的一种推广，允许在构成逻辑模式图中树结构的标识代理之上建立索引。访问支持关系（ASR）是对连接索引的另一种推广 [12]，它允许在逻辑模式节点构成的任意链的实例上定义索引。这一方案灵活性更高，允许定义既存储每条链的完整实例、也存储其部分实例的索引。除了最后这一特性之外，层次连接索引和 ASR 的内容都可以表示为 psj 查询，因而都可以定义为 gmap。但由于 gmap 查询不支持并运算，它们无法表示外连接，因此也无法存储链的不完整实例。

With respect to the translation algorithm, our work most closely resembles research at the University of Waterloo on materialized views [3, 28]. Our algorithm supports a more restricted query language, but uses information about inclusion and functional dependencies as well as “topological” information implicit in a graph-based logical schema. This information allows us to identify solutions that would be missed by the more general algorithm. Section 5 contains an example of a solution that can only be found when inclusion dependencies are taken into account. Similarly, our

handling of functional and multivalued dependencies is more general than that of the algorithm of Yang and Larson, which simply uses the primary key information for each relation. Unless all non-trivial dependencies are generated by superkeys (i.e., unless all relations are in at least 4th Normal Form), our scheme will find more solutions.

在翻译算法方面，与我们的工作最接近的是滑铁卢大学关于物化视图的研究 [3, 28]。我们的算法支持的查询语言更受限，但它利用了包含依赖和函数依赖的信息，以及基于图的逻辑模式中隐含的“拓扑”信息。借助这些信息，我们能找出那些更通用的算法会漏掉的解。第 5 节中就有一个只有考虑包含依赖才能找到的解的例子。类似地，我们对函数依赖和多值依赖的处理，比 Yang 和 Larson 的算法更具一般性，后者只利用了每个关系的主键信息。除非所有非平凡依赖都由超键产生（即除非所有关系都至少满足第四范式），否则我们的方案会找到更多的解。

With respect to the integration with the rest of the query optimizer, most earlier efforts use a two-stage approach, where the queries are first translated into queries over physical structures, and the resulting queries are then optimized one-by-one by a conventional optimizer. In addition to the work on materialized views [3, 28], such efforts include research whose goal was not physical data independence but simply processing efficiency. Examples include research on reusing common subexpressions within a query [9] or between multiple queries [22], reusing results of previous queries [8], and using integrity constraints for semantic query optimization [6]. Kemper and Moerkotte [13] opt for a unified approach of translation and optimization for the ASRs by extending a rule based optimizer to include appropriate rewriting rules. Our approach of enhancing a conventional optimizer with the necessary translation steps takes advantage of full

cost information available to the optimizer to perform early pruning of inferior solutions, while keeping the overall optimization cost low.

在与查询优化器其余部分的集成方面，大多数早期工作采用两阶段的做法：先把查询翻译成针对物理结构的查询，再由传统优化器逐个优化所得的查询。除了关于物化视图的工作 [3, 28] 之外，这类工作还包括一些并不以物理数据独立性为目标、而只是为了提高处理效率的研究，例如：在一个查询内部 [9] 或多个查询之间 [22] 复用公共子表达式，复用先前查询的结果 [8]，以及利用完整性约束进行语义查询优化 [6]。Kemper 和 Moerkotte [13] 则为 ASR 选择了翻译与优化统一的做法，他们扩展了一个基于规则的优化器，加入相应的改写规则。我们的做法是在传统优化器中加入必要的翻译步骤，这样可以利用优化器掌握的全部代价信息，及早剪掉较差的解，同时使整体的优化代价保持在较低水平。

10 Conclusions

10 结论

We have presented a new approach to physical schema design that uses a declarative language to describe the contents of storage structures. Carefully restricting the language allows efficient algorithms to translate queries over the logical schema into access plans using the physical data structures. We have shown how to integrate the query translation algorithm into a conventional query optimizer. A simple modification of the query translation algorithm supports propagation of updates to the database. A prototype system that incorporates the major aspects of this approach is currently operational. We have used it to demonstrate in a realistic environment how our approach can achieve significant performance gains over more conventional ones.

我们提出了一种新的物理模式设计方法，它用一种声明式语言来描述存储结构的内容。通过对这种语言加以精心限制，可以用高效的算法把针对逻辑模式的查询翻译成使用物理数据结构的访问计划。我们展示了如何把查询翻译算法集成到传统的查询优化器中。对查询翻译算法稍作修改，就能支持把更新传播到数据库。一个实现了该方法主要内容的原型系统目前已经可以运行。我们用它在一個贴近实际的环境中证明了：与较传统的方法相比，我们的方法能获得显著的性能提升。

In the future, we plan to extend the translation algorithm to take into account additional integrity constraints and also permit the translation of queries into unions of other queries. We would also like to incorporate storage structures that offer a hierarchical interface, and study the feasibility of using the query optimizer to exploit in-memory data storage structures. Finally, the increase of available choices in the physical schema design puts the burden on the database administrator to make the correct choices. We need to design tools that guide the administrator in choosing the appropriate combination of storage structures.

今后，我们计划扩展翻译算法，使其能考虑更多的完整性约束，并允许把查询翻译成其他查询的并。我们还希望纳入提供层次化接口的存储结构，并研究利用查询优化器来使用内存数据存储结构的可行性。最后，物理模式设计中可选方案的增多，把做出正确选择的负担加在了数据库管理员身上。我们需要设计一些工具，指导管理员选择合适的存储结构组合。

References

参考文献

timization for the ASRs by extending a rule based optimizer to include appropriate rewriting rules. Our approach of enhancing a conventional optimizer with the necessary translation steps takes advantage of full cost information available to the optimizer to perform early pruning of inferior solutions, while keeping the overall optimization cost low.

- [8] S. Finkelstein. Common expression analysis in database applications. In *Proc. of the ACM SIGMOD*, 1982.
- [9] P. Hall. Optimization of a single relational expression in a RDBMS. *IBM Journal of Research and Development*, 20(3), May 1976.
- [10] Y. Ioannidis, M. Livny, E. Haber, R. Miller, O. Tsatalos, and J. Wiener. Desktop Experiment Management. *IEEE*

10 Conclusions

We have presented a new approach to physical schema design that uses a declarative language to describe the contents of storage structures. Carefully restricting the language allows efficient algorithms to translate queries over the logical schema into access plans using the physical data structures. We have shown how to integrate the query translation algorithm into a conventional query optimizer. A simple modification of the query translation algorithm supports propagation of updates to the database. A prototype system that incorporates the major aspects of this approach is currently operational. We have used it to demonstrate in a realistic environment how our approach can achieve significant performance gains over more conventional ones.

In the future, we plan to extend the translation algorithm to take into account additional integrity constraints and also permit the translation of queries into unions of other queries. We would also like to incorporate storage structures that offer a hierarchical interface, and study the feasibility of using the query optimizer to exploit in-memory data storage structures. Finally, the increase of available choices in the physical schema design puts the burden on the database administrator to make the correct choices. We need to design tools that guide the administrator in choosing the appropriate combination of storage structures.

References

- [1] A. Aho, Y. Sagiv, and J. Ullman. Equivalences Among Relational Expressions. *SIAM Journal of Computing*, 8(2):218–247, 1979.
- [2] E. Bertino and W. Kim. Indexing Techniques for Queries on Nested Objects. *IEEE Transactions on Knowledge and Data Engineering*, 1(2):196–214, June 1989.
- [3] J. Blakeley, N. Coburn, and P. Larson. Updating derived relations. *ACM Transactions on Database Systems*, 14(3):369–400, Sept. 1989.
- [4] M. Carey, D. DeWitt, M. Franklin, N. Hall, M. McAuliffe, J. Naughton, D. Schuh, M. Solomon, C. Tan, O. Tsatalos, S. White, and M. Zwilling. Shoring Up Persistent Applications. In *Proc. of the ACM SIGMOD*, May 1994.
- [5] M. Carey, D. DeWitt, and S. Vandenberg. A Data Model and Query Language for Exodus. In *Proc. of the ACM SIGMOD*, pages 413–423, Chicago, IL, June 1988.
- [6] U. Chakravarthy. Logic-Based Approach to Semantic Query Optimization. *ACM Transactions on Database Systems*, 15(2):163–207, June 1990.
- [7] A. Chandra and P. Merlin. Optimal implementation of conjunctive queries in relational data bases. In *Proceedings of Annual ACM Symposium on Theory of Computing*, pages 77–90, May 1977.
- [8] *Data Engineering Journal*, 16(1):19–23, Mar. 1993.
- [9] K. Kato and T. Masuda. Persistent Caching. *IEEE Transactions on Software Engineering*, 18(7), July 1992.
- [10] A. Kemper and G. Moerkotte. Access Support in Object Bases. In *Proc. of the ACM SIGMOD*, 1990.
- [11] A. Kemper and G. Moerkotte. Advanced Query Processing in Object Bases Using Access Support Relations. In *Proc. of the Int'l VLDB Conf.*, pages 290–301, Brisbane, Australia, 1990.
- [12] W. Kim, E. Bertino, and J. Garza. Composite Objects Revisited. In *Proc. of the ACM SIGMOD*, 1989.
- [13] W. Kim, J. F. Garza, N. Ballou, and D. Woelk. Architecture of the ORION Next-generation Database System. *IEEE Transactions on Knowledge and Data Engineering*, 2:109–124, Mar. 1990.
- [14] D. Maier and J. Stein. Indexing in an Object-Oriented DBMS. In *2nd Int'l Workshop on Object-Oriented Database Systems*, pages 171–182, Asilomar, CA, Sept. 1986.
- [15] R. Miller, Y. Ioannidis, and R. Ramakrishnan. The Use of Information Capacity in Schema Integration and Translation. In *Proc. of the Int'l VLDB Conf.*, 1993.
- [16] J. Orenstein, S. Haradhvala, B. Margules, and D. Sakahara. Query Processing in the ObjectStore Database System. In *Proc. of the ACM SIGMOD*, San Diego, CA, 1992.
- [17] N. Roussopoulos. View Indexing in Relational Database. *ACM Transactions on Database Systems*, 7(2), June 1982.
- [18] N. Roussopoulos, N. Economou, and A. Stamenas. ADMS: A Testbed for Incremental Access Methods. *IEEE Transactions on Knowledge and Data Engineering*, 5(5):762–773, Oct. 1993.
- [19] P. Selinger, M. Astrahan, D. Chamberlin, R. Lorie, and T. Price. Access Path Selection in a Relational Database Management System. In *Proc. of the ACM SIGMOD*, pages 23–34, 1979.
- [20] T. Sellis. Global Query Optimization. In *Proc. of the ACM SIGMOD*, pages 191–205, 1986.
- [21] E. Shekita and M. Carey. Performance Enhancement Through Replication in an Object-Oriented DBMS. In *Proc. of the ACM SIGMOD*, pages 325–336, 1989.
- [22] O. Tsatalos and Y. Ioannidis. A Unified Framework for Indexing in Database Systems. In *Int'l Conf. on Database and Expert System Applications*, Sept. 1994.
- [23] O. Tsatalos, M. Solomon, and Y. Ioannidis. Enhanced Storage Structures for OODBMSs. Unpublished manuscript, available from the authors, Feb. 1994.
- [24] P. Valduriez. Join Indices. *ACM Transactions on Database Systems*, 12(2):218–246, June 1987.
- [25] P. Valduriez, S. Khoshafian, and G. Copeland. Implementation Techniques of Complex Objects. In *Proc. of the Int'l VLDB Conf.*, pages 101–109, Kyoto, Japan, 1986.
- [26] H. Yang and P. Larson. Query transformation for PSJ-queries. In *Proc. of the Int'l VLDB Conf.*, 1987.